UNIVERSITY OF CALGARY

GoCycle

SENG 300 | Fall 2026

Assignment 1: Requirements, Design, and Implementation

TEAM MEMBER	UCID	UNIVERSITY EMAIL
Person A - Nazeeh Hammad	30173895	nazeeh.hammad@ucalgary.ca
Person B - Toby Allan	30245907	toby.allan@ucalgary.ca
Person C - Juan Pablo Padilla	30229022	juanpablo.padilla@ucalgary.ca

Repository: github.com/padillajuanpablo434-design/Assignment-1

Prepared for: SENG 300, Department of Computer Science

This report describes the completely proposed service and identifies the Java prototype features currently implemented. The diagrams illustrate the group design; some client-facing features remain outside the implemented core.

Contents

1. Requirements classification

2. Use case model

3. UML class design

4. Design rationale (one page)

5. Design patterns

6. Implementation coverage and limitations

7. Sources and external assistance

Project overview

GoCycle is a proposed citywide shared bicycle and scooter service. Riders rent vehicles from stations and return them to any dock with capacity. Fare rules differ for casual, member and student riders; electric vehicles require sufficient battery; and faults must notify operations and maintenance while preventing unsafe rental.

Our team modeled the client request, prepared a use case and class design, and built a Java 17 Maven prototype. The source is organized by responsibility: **rider**, **pricing**, **trip**, **vehicle**, **station**, **payment**, **fault** and **app**.

1. Requirements classification

Functional requirements describe system behaviours; quality requirements describe desired system attributes or measurable acceptance properties; constraints limit the solution or operating environment. The following IDs preserve the classification in the group requirements model. Several measurable quality statements are team interpretations of the client brief.

Trips and pricing

ID	Functional requirement
FR-01	Start a rental by taking an available vehicle at a station.
FR-02	Identify a rider and begin a trip via app or station screen/card reader.
FR-03	Support casual pay-per-trip riders.
FR-04	Support monthly members with a lower per-minute rate.
FR-05	Support students with a reduced rate.
FR-06	Calculate unlock fee plus per-minute fare using rider type.
FR-07	Add surcharge for trips longer than two hours.
FR-08	Place refundable deposit hold at trip start.
FR-09	Release deposit hold at trip end.

Vehicles

ID	Functional requirement
FR-10	Provide pedal bike, e-bike and e-scooter categories.
FR-11	Track battery levels for electric vehicles.
FR-12	Reject electric vehicle rental below 20% battery.

Stations and returns

ID	Functional requirement
FR-13	Represent stations and their docking points.
FR-14	Allow return to any station with a free dock.
FR-15	End a trip only when its vehicle is confirmed locked.
FR-16	When full, direct rider to nearest station with a free dock.
FR-17	Keep trip active and charges running until successful lock.

Faults and maintenance

ID	Functional requirement
FR-18	Accept rider reports of vehicle faults.
FR-19	Allow electric vehicles to self-report low battery and faults.
FR-20	Remove broken vehicles from service.
FR-21	Notify maintenance and control-room operators of reported faults.

Payment and zones

ID	Functional requirement
FR-22	Support credit card, city transit card and phone wallet.
FR-23	Enforce configured allowed payment methods by zone.

Accessibility and branding

ID	Functional requirement
FR-24	Offer rider-facing screens in multiple languages.
FR-25	Offer read-aloud information on rider-facing screens.
FR-26	Display configurable zone colours and logos.

Operations and receipts

ID	Functional requirement
FR-27	Show operators state of stations and vehicles.
FR-28	Support operator rebalancing across stations.
FR-29	Allow operators to take vehicles out of and back into service.
FR-30	Issue a receipt after payment.
FR-31	Deliver receipt through app or email.

Quality requirements

ID	Quality requirement
QR-01	New rider and pricing categories are extendable without altering existing fare or trip logic.
QR-02	Prices add up precisely to the cent, avoiding floating-point rounding.
QR-03	A trip cannot complete without correct lock confirmation; billed duration respects that event.
QR-04	Vehicles with open faults cannot be rented.
QR-05	Screen information is offered through all supported languages and read-aloud.
QR-06	Zone colours/logo are changeable through configuration rather than code edits.

Constraints

ID	Constraint
C-01	The client specifies phone app and station screen/card reader access channels.
C-02	Each docking point can accommodate at most one vehicle.
C-03	The named payment types are credit card, transit card and phone wallet.
C-04	Zone branding must come from configuration.

Optional proposals from the group requirements document, not binding client requirements: live station-availability map; short-term reservation holds; battery/range filter; and a fare estimate before unlocking.

2. Use case model

The primary human actors are **Rider** and **Operator**. Supporting actors are **Maintenance Crew**, **Payment Provider**, and the telemetry-capable **Electric Vehicle**. The diagram distinguishes included steps from optional redirect behaviour.

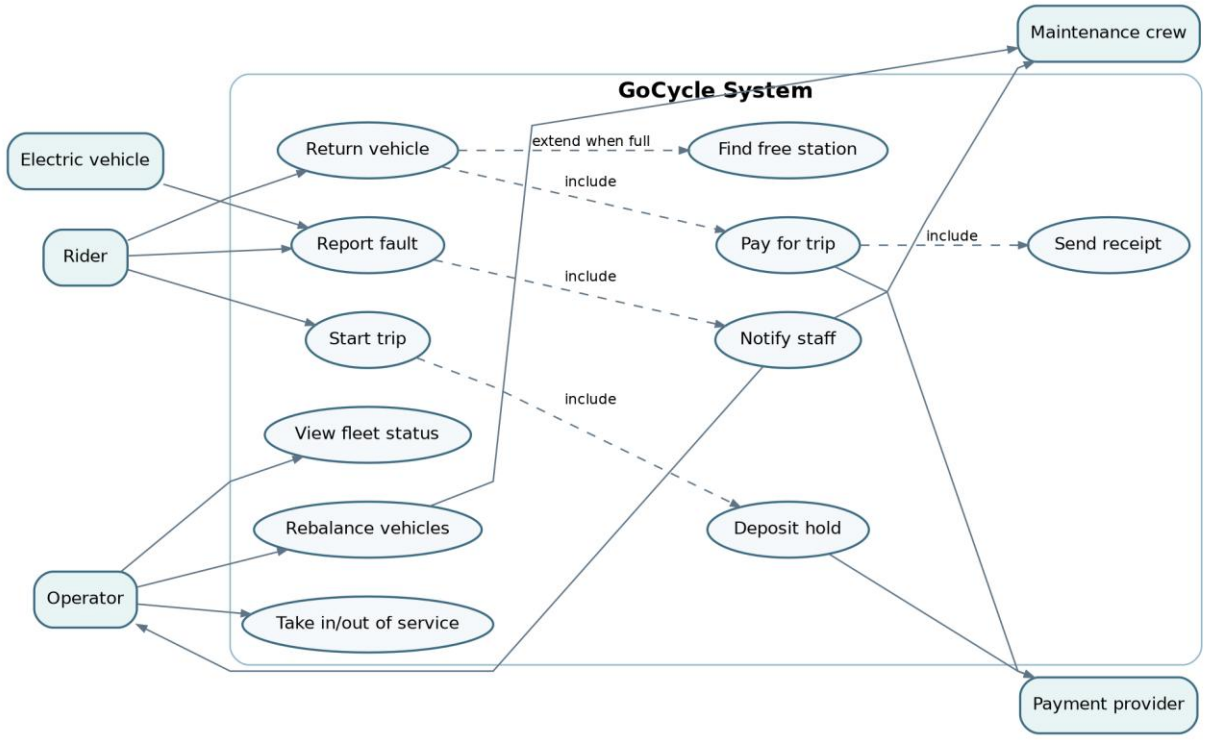


Figure 1. Proposed GoCycle use case model (simplified for report readability).

Key relationships: **Start Trip** includes deposit hold; **Return Vehicle** includes payment, and may trigger nearest-free-station redirection; **Pay for Trip** includes receipt delivery; **Report Fault** includes notifying maintenance and operators.

UC-01 - Start Trip

Primary actor: Rider. **Secondary:** Payment Provider. **Preconditions:** a registered rider, an available docked vehicle, and an eligible payment method.

1. Rider identifies themselves using a supported access channel and selects a vehicle.

2. System verifies availability and, for electric vehicles, battery at least 20%.

3. System checks zone restrictions and requests a refundable $50.00 deposit hold.

4. On approval, the vehicle is unlocked/assigned, marked IN_USE, and the trip start time is recorded.

Alternative paths: unavailable or low-battery vehicle; disallowed payment method; refused deposit; failed unlock. The rider chooses an alternative or the request ends without an active rental.

Postcondition: trip ACTIVE with hold placed and start recorded; otherwise the vehicle remains docked and no hold is outstanding. **Related:** FR-01, 02, 08, 11, 12, 22, 23.

UC-02 - Return Vehicle

Primary actor: Rider. **Precondition:** active trip.

1. Rider arrives at any station and checks for a free docking point.

2. If full, system directs rider toward the nearest station with capacity; the trip continues.

3. Rider docks the vehicle, and the system verifies the correct vehicle is physically locked.

4. System determines end time and fare, collects payment, releases hold and sends the receipt.

Alternative paths: lock not confirmed, wrong vehicle, full station, or failed settlement. Charging cannot stop before a valid return lock.

Postcondition: vehicle secured at a station; settled trip complete with a receipt. **Related:** FR-13 to FR-17, FR-09 and FR-30.

UC-03 - Report Fault

Primary actor: Rider or electric vehicle. **Secondary actors:** maintenance crew and operator. **Precondition:** known vehicle.

1. Rider selects a vehicle and fault type and optionally adds details, or an electric vehicle raises a telemetry report.

2. The system records FaultReport, marks vehicle OUT_OF_SERVICE and prevents future rentals.

3. FaultService notifies registered FaultListener observers, including MaintenanceCrew and OperatorConsole.

4. The reporting rider receives acknowledgement when a user-facing channel is available.

Alternative paths: duplicate fault, a fault during an active trip, or a self-reported low battery. The damaged vehicle remains unavailable after return. **Related:** FR-18 to FR-21, QR-04.

UC-04 - Pay for Trip

Primary actor: Rider. **Secondary:** Payment Provider. **Precondition:** trip has returned with confirmed lock.

1. Calculate billable minutes, rounded up to full minutes.

2. Use rider PricingStrategy for unlock fee, time charge and any over-120-minute surcharge.

3. Charge using an allowed PaymentMethod and release the refundable deposit hold.

4. Create a detailed receipt and deliver by the selected app/email channel.

Alternative paths: charge decline, alternate allowed payment, or receipt delivery failure. Pending financial settlement is a known implementation limitation of this Java prototype. **Related:** FR-03 to FR-09, FR-22, 23, 30, 31.

UC-05 - Rebalance Vehicles

Primary actor: Operator. **Secondary:** Maintenance Crew. **Precondition:** operator is authorized to view fleet status.

1. Operator inspects station occupancy, vehicle availability and battery state.

2. Operator selects vehicles to relocate and a destination with enough free docks.

3. Crew moves vehicles, docks them securely and obtains lock confirmation.

4. System updates station inventories and restores eligible vehicles to service.

Alternative paths: destination fills up, vehicle is already rented, a fault is discovered, or the new dock fails to lock. The proposed workflow is modeled, not fully implemented as a control-room UI. **Related:** FR-27 to FR-29.

3. UML class design

The class model contains substantially more than eight classes, including inheritance among riders and vehicles, focused interfaces for payment and pricing, a vehicle factory creator hierarchy, Observer notifications, station composition, and the associations needed for a trip.

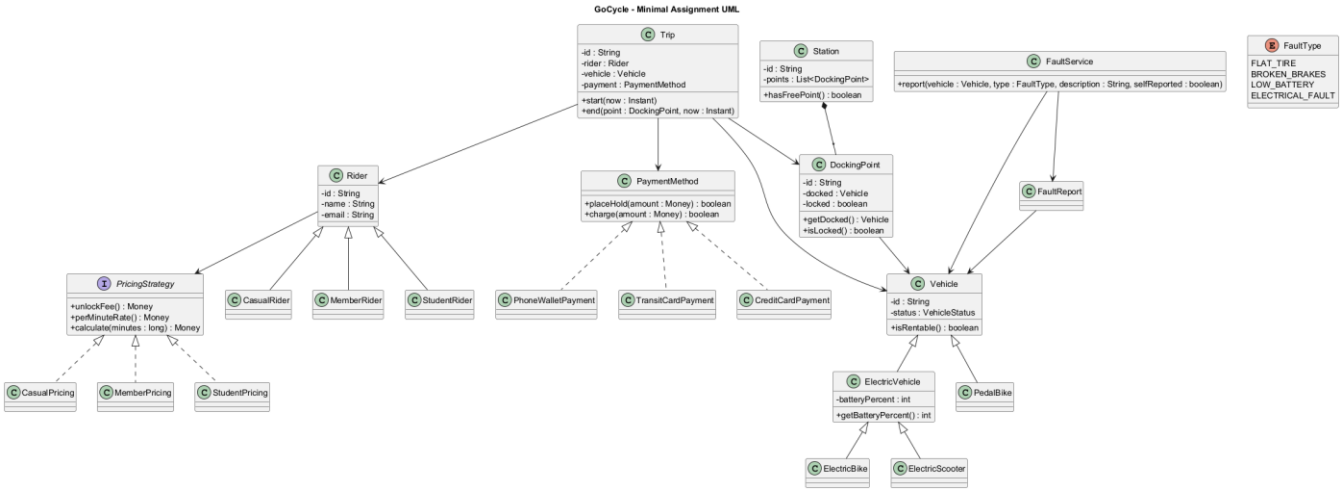


Figure 2. Integrated class relationships after the three group contributions. See source diagram in docs/complete-class-diagram.puml.

4. Design rationale (one page)

GoCycle separates rider identity, pricing, rental lifecycle, vehicles, docking, payments and maintenance notifications because they change for different reasons. **Trip** coordinates one rental but delegates cost calculation to **PricingStrategy**, payment actions to **PaymentMethod** and lock verification to **DockingPoint**. **StationFinder** provides a distinct station-availability and distance helper. This allows the core prototype to remain flexible as supported vehicles, fares and integrations evolve.

SOLID principles in the implemented design

Single Responsibility (SRP): Trip owns trip state; pricing strategies calculate fares; StationFinder searches free stations; FaultService records and broadcasts vehicle faults; maintenance and operator listeners record different kinds of notifications.

Open/Closed (OCP): New rider tariffs can implement PricingStrategy and new fault destinations can implement FaultListener. VehicleCreator subclasses encapsulate vehicle construction, though registering a new enum vehicle type still requires factory-registry changes.

Liskov Substitution (LSP): All rider subclasses follow the Rider contract, and all vehicle subclasses provide isRentable(). An electric vehicle adds a battery restriction without changing the caller-facing method signature. Payment alternatives implement shared hold/charge/release semantics.

Interface Segregation (ISP): FaultListener provides only onFault(), ReceiptSender only send(), and PricingStrategy has a small fare-calculation contract. Clients do not implement unrelated capabilities.

Dependency Inversion (DIP): Trip depends on PricingStrategy and PaymentMethod interfaces rather than a concrete fare calculator or card. FaultService depends on FaultListener; vehicle creation can be requested via VehicleFactory.

Patterns and maintainability

Strategy selects casual, member or student rates. **Decorator** applies the same surcharge to any strategy. **Observer** notifies multiple consumers of a fault without embedding those recipients in the reporting service. **Factory Method** is implemented through concrete VehicleCreator subclasses, while a VehicleFactoryImpl registry chooses which creator to use. These patterns add types and indirection but improve extensibility and testability.

Safety and practical trade-offs

The rental system preserves OUT_OF_SERVICE when a faulty vehicle is returned. The battery monitor can mark electric vehicles unavailable and send fault alerts. Money uses integer cents; station selection uses geographical distance. The implementation is an educational console prototype: multilingual UI, speech, actual lock sensors, third-party payments and road routing are not deployed. Financial settlement after a physically locked return needs a richer pending-payment state in a future iteration.

5. Design patterns

5.1 Strategy - rider pricing (Person A)

Problem. Casual, monthly member and student riders pay different rates, and the client wants future rider categories. Putting rider-type conditionals in Trip would couple rental orchestration to fare policy.

Choice and justification. PricingStrategy defines unlockFee(), perMinuteRate() and calculate(). Each concrete strategy supplies its own values; a Rider receives one strategy. Trip calls the abstraction and does not branch on rider type.

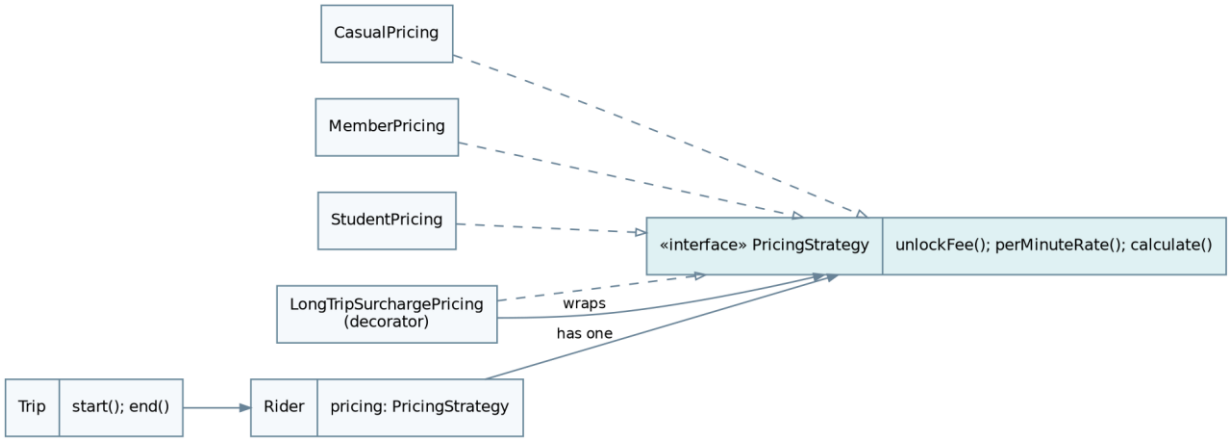


Figure 3. Pricing Strategy and the surcharge decorator (simplified UML).

Rider category	Unlock	Per minute
Casual	$1.50	$0.35
Member	$0.00	$0.20
Student	$1.00	$0.25

Benefits. New price calculations can be unit-tested independently; responsibility is localized; an additional fare can be introduced with minimal changes to Trip. **Trade-offs.** More objects and a separate mapping from rider type to pricing implementation.

Supporting Decorator - long-trip surcharge

LongTripSurchargePricing wraps PricingStrategy and adds $5.00 only when a trip lasts more than 120 billable minutes. The wrapper is also a PricingStrategy, so it works uniformly for all rider categories. This avoids copying the same surcharge logic into each concrete strategy.

5. Design patterns (continued)

5.2 Factory Method - vehicle construction (Person B)

Problem. Creating pedal bikes, e-bikes and scooters directly throughout the application would tie calling code to concrete subclasses. Vehicle types may expand later.

Choice and justification. The abstract VehicleCreator declares the factory method createVehicle(id), with PedalBikeCreator, ElectricBikeCreator and ElectricScooterCreator selecting their corresponding products. VehicleFactoryImpl is a registry that delegates creation to the appropriate creator.

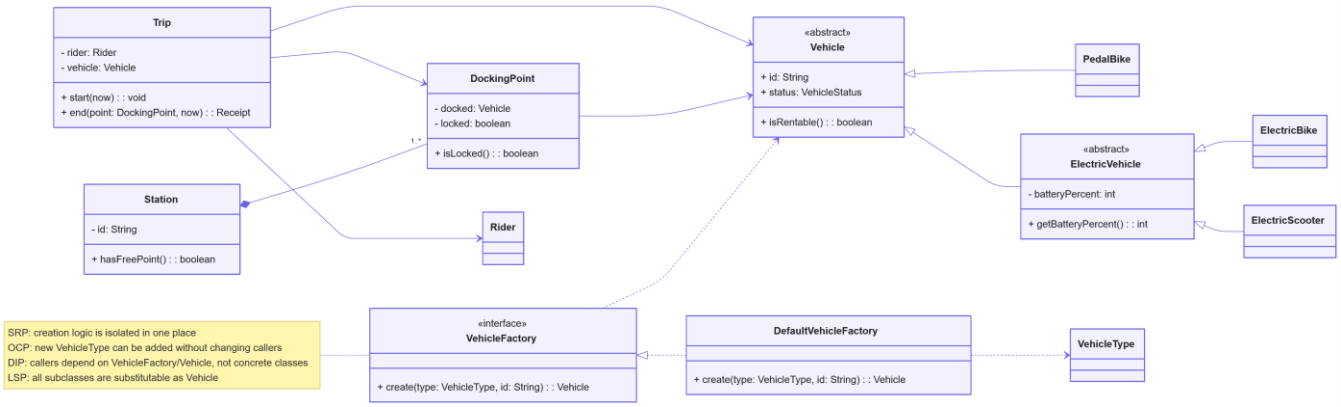


Figure 4. Factory Method creators and the registry (simplified UML).

Benefits. Object construction and product-specific defaults are isolated; callers can work with Vehicle abstractions; creator implementations are independently testable. **Trade-offs.** An expanded type hierarchy and registration steps for new VehicleType values. The registry is not itself the Factory Method; the creator subclasses provide that pattern.

5.3 Observer - fault notification (Person C)

Problem. Faults must reach at least two separate receivers, maintenance and the control-room operator. Hard-coded concrete dependencies would make additional alert recipients difficult to add.

Choice and justification. FaultService is the subject; FaultListener defines onFault(FaultReport); MaintenanceCrew and OperatorConsole subscribe as observers. A submitted fault is recorded, its vehicle becomes OUT_OF_SERVICE and subscribers receive the event. ElectricFaultMonitor connects vehicle telemetry to this service.

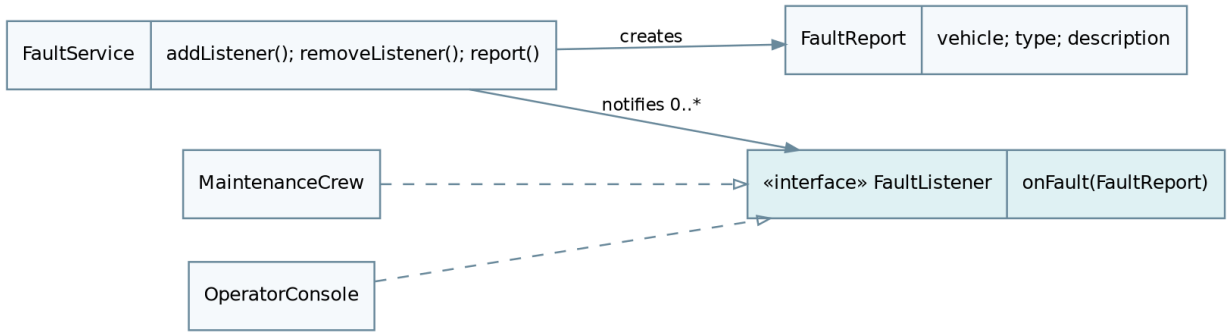


Figure 5. Observer pattern used for fault reporting (simplified UML).

Benefits. Independent alert destinations, extension through new listeners, and isolated tests of notification behaviour. **Trade-offs.** Listener registration/lifecycle handling, synchronous notifications and a need to consider repeat reports, errors and future asynchronous delivery.

6. Implementation coverage and limitations

The full requirements model describes the intended GoCycle service; the Java source implements its central domain logic rather than a deployed mobile and kiosk solution.

Area	Coverage	Clarification
Fare and trip lifecycle	Implemented core	Pricing strategies, surcharge, holds, charge, receipts and return-lock invariant.
Three vehicle types	Implemented core	Pedal, e-bike, scooter; electric eligibility requires battery >=20%.
Stations and return	Implemented core	Docking/lock check and a straight-line nearest-free-station helper.
Faults and alerts	Implemented core	Observer, operator/maintenance listeners, registered electric battery monitor.
Three payment methods	Simulation	Allowed types enforced by a zone wrapper; simulated in-memory charges.
Accessible mobile/kiosk UI	Design only	Languages, audio, physical screens/card readers and branding not built.
Operator fleet management	Partial	Fault alerts available; full dashboard and rebalancing controls not built.
Transaction failures	Known limitation	If charge fails after physical return, ACTIVE remains; a payment-pending state is advisable.

Verification basis

The group tested its integrated branch using Maven on Java 17. A local run reported **54 tests, 0 failures, 0 errors, 0 skipped**, and the Main demonstration completed successfully. The merged main branch should be tested once more before packaging; this statement describes the recorded local test run, not a new independent run performed for this document.

Suggested improvements (not required by the client)

Show live station availability; allow a short temporary reservation; provide an electric battery-range filter; and show an up-front fare estimate. These were proposed in the group requirements model as future enhancements, not implemented features.

7. Sources and external assistance

Primary course source: University of Calgary, SENG 300 - Fall 2026, Assignment 1: Requirements, Design, and Implementation (instructor-provided specification).

Project source: GoCycle team Git repository, https://github.com/padillajuanpablo434-design/Assignment-1, including the integrated main branch, README, requirements/use cases, class diagram, pattern write-ups and Java source. The report adapts these team-authored materials into one submission.

External resources and AI assistance: We used AI to brainstorm design ideas and improve the structure, wording, and formatting of our report.